Listing 32.12 An extension method for currency validation

```
public static class ValidationExtensions
{
    public static IRuleBuilderOptions<T, string>           ❶
        MustBeCurrencyCode<T>(                             ❶
            this IRuleBuilder<T, string> ruleBuilder)      ❶
    {
        return ruleBuilder                                 ❷
            .Must(value => _allowedValues.Contains(value)) ❷
            .WithMessage("Not a valid currency code");     ❷
    }

    private static readonly string[] _allowedValues =      ❸
        new []{ "GBP", "USD", "CAD", "EUR" };              ❸
}
```

❶ Creates an extension method that can be chained from RuleFor() for string properties

❷ Applies the same validation logic as before

❸ The currency code values to allow

You can then update your `CurrencyConverterModelValidator` to use the new extension method, removing the duplication in your validator and ensuring consistency across country-code fields:

```
RuleFor(x => x.CurrencyTo)
    .NotEmpty()
    .Length(3)
    .MustBeCurrencyCode();
```

Another advantage of the FluentValidation approach of using standalone validation classes is that they are created using DI, so you can inject services into them. As an example, consider the `[CurrencyCode]` validation attribute from listing 32.9, which used a service, `ICurrencyProvider`, from the DI container. This requires using